We don't need to check HDFS data to know how it is stored. Since we have imported it from MySQL, we are aware of the schema. It is almost compatible with Hive except for some data types.

Once the database is modelled, we can load the extracted data for cleaning. Still, the data in the table is de-normalised. Aggregate the required columns from the different tables.

Open a Hive or Beeline shell, or if you have HUE (Hadoop User Experience) installed, do open the Hive editor and run the commands given below:

```
CREATE DATABASE <database name>;
USE <database name>;
CREATE TABLE <table name>
(
variable1 DATATYPE ,
variable2 DATATYPE ,
variable3 DATATYPE ,
. ,
. ,
. ,
variable DATATYPE
)
PARTITIONED BY (Year INT, Month INT )
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

LOAD DATA INPATH 'hdfs://Hadoop_sahara_local/user/rkkrishnaa/
database/files.csv'
```

Similarly, we can aggregate the data from the multiple tables with the 'OVERWRITE INTO TABLE' statement.

Hive supports partitioning tables to improve the query performance by distributing the execution load horizontally. We prefer to partition the columns that store the year and month. Sometimes, a partitioned table creates more tasks in a MapReduce job.

The load phase

We have checked the transform phase earlier. It is time to load the transformed data into a data warehouse directory in HDFS, which is the final state of the data. Here we can apply our SQL queries to get the appropriate results.

All DML commands can be used to analyse the warehouse data based on the use case *(https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DML)*.

Results can be downloaded as CSV, graphs or charts for analysis. They can be integrated with other popular BI tools such as Talend OpenStudio, Tabelau, etc.

What next?

We had a brief look at the benefits of OpenStack Sahara and explored a typical example of an ETL task in the Hadoop ecosystem. It is time to automate the ETL job with the Oozie workflow engine.

If you are experienced in using Mistral, you can stick with it. Most Hadoop users are acquainted with Apache Oozie, so I am using Oozie for this example.

Step 1: Create a *job.properties* file, as follows:

```
nameNode=hdfs://hadoop_sahara_local:8020
jobTracker=<rm-host>:8050
queueName=default
examplesRoot=examples
oozie.use.system.libpath=true
oozie.wf.application.path=${nameNode}/user/${user.name}
oozie.libpath=/user/root
```

Step 2: Create a *workflow.xml* file to define the tasks for extracting data from the MySQL data store to HDFS, as follows:

```
<workflow-app xmlns="uri:oozie:workflow:0.2" name="my-etl-job">
<global>
<job-tracker>${jobTracker}</job-tracker>
<name-node>${nameNode}</name-node>
</global>

<start to="extract"/>
<action name="extract">
<sqoop xmlns="uri:oozie:sqoop-action:0.2">
<configuration>
<property>
<name>mapred.job.queue.name</name>
<value>${queueName}</value>
</property>
</configuration>
<command>import --connect jdbc:mysql://<database
host>:3306/<database name> --username <database user>
--password <database password> --table <table name> --driver
com.mysql.jdbc.Driver --target-dir hdfs://hadoop_sahara_
local/user/rkkrishnaa/database${date} --m 3 </command>
</sqoop>

<ok to="transform"/>
<error to="fail"/>

<action name="transform">
<hive xmlns="uri:oozie:hive-action:0.2">
<configuration>
<property>
<name>mapred.job.queue.name</name>
<value>${queueName}</value>
</property>
<property>
<name>oozie.hive.defaults</name>
<value>/user/rkkrishnaa/oozie/hive-site.xml</value>
</property>
</configuration>
```